

Modelos Generativos Profundos

Clase 7: Modelos autorregresivos y LLMs (parte III)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Tokenizadores

Positional encoding

Mecanismos de normalización

Mecanismos de atención

Mixture of experts

Tokenizadores

Problema de tokenización

Dado un corpus de entrenamiento, es necesario elegir un **algoritmo de tokenización**, el cual definirá el vocabulario $\mathcal{V}$ sobre el que se tokenizará dicho corpus.

- Aumentar $|\mathcal{V}|$ aumenta la cantidad de parámetros del modelo.
- Disminuir $|\mathcal{V}|$ provoca que las secuencias de tokens sean más largas, lo cual aumenta el costo de procesamiento.

Encontrar la *mejor* tokenización es un problema NP-completo, por lo que se debe recurrir a métodos aproximados.

Byte pair encoding

Uno de los algoritmos de tokenización más usuales es el algoritmo **byte pair encoding (BPE)**:

- Comienza considerando cada caracter como un token de $\mathcal{V}$.
- Luego, forma nuevos tokens combinando cadenas de caracteres de acuerdo a su frecuencia de ocurrencia en el corpus.
- Para esto, calcula la frecuencia de todos los bigramas en el corpus y agrega al vocabulario el bigrama con mayor frecuencia.
- Este proceso se repite hasta que se alcanza el tamaño de vocabulario deseado.
- Otros algoritmos como **WordPiece** siguen enfoques similares.

Tokens especiales

Es usual incluir **tokens especiales** que cumplen funciones específicas en el modelo. Algunos ejemplos son:

- **<SEP>**: usado para separar dos partes de la secuencia.
- **<CLS>**: usado como embedding de la secuencia completa.
- **<MASK>**: usado para enmascarar partes de la secuencia.
- **<TOOL>**: usado para indicar el uso de herramientas externas.
- **<SYSTEM>**, **<USER>**, **<ASSISTANT>**: usado para indicar roles en una conversación.

Por otro lado, es usual fijar $|\mathcal{V}|$ cercano a un múltiplo de 64, lo que suele agregar tokens vacíos. Estos tokens vacíos luego se pueden usar, mediante fine tuning, como tokens especiales.

Positional encoding

Embeddings posicionales absolutos

La información inyectada por los embeddings posicionales influye directamente en el mecanismo de atención:

$$y_i = \sum_{j=1}^T a_{ij} v_j, \quad \text{donde} \quad a_{ij} = \frac{\exp\left(\frac{\langle q_i, k_j \rangle}{\sqrt{P}}\right)}{\sum_{t=1}^T \exp\left(\frac{\langle q_i, k_t \rangle}{\sqrt{P}}\right)}$$

Cada vector $q_t, k_t, v_t \in \mathbb{R}^P$ es calculado como una proyección $x_t \mapsto W(x_t + p_t)$ del embedding $x_t \in \mathbb{R}^D$ al espacio $\mathbb{R}^P$, donde el vector $p_t \in \mathbb{R}^D$ es un **embedding posicional absoluto**.

Distintas formas de definir los embeddings $p_1, \dots, p_T \in \mathbb{R}^D$ definen distintos mecanismos de positional encoding.

Embeddings posicionales sinusoidales

Si bien se puede utilizar una matriz de embedding, otra opción es utilizar **embeddings sinusoidales**:

$$p_t = \begin{pmatrix} \sin(\omega_1 t) \\ \cos(\omega_1 t) \\ \vdots \\ \sin(\omega_{D/2} t) \\ \cos(\omega_{D/2} t) \end{pmatrix} \in [-1, 1]^D$$

- Las **frecuencias de oscilación**, $\omega_1, \dots, \omega_{D/2} > 0$, suelen elegirse siguiendo una progresión geométrica decreciente.
- Es decir, $\omega_k = b^{-\frac{k}{D}}$, donde $b > 1$ es un hiperparámetro.
- Notar que este mecanismo no utiliza parámetros entrenables.

Embeddings posicionales relativos

También es posible utilizar **embeddings relativos**, los cuales codifican la distancia relativa entre los tokens de la secuencia:

- Para el cálculo de atención de un token $q_i \in \mathbb{R}^P$ sobre un par key-value $(k_j, v_j) \in \mathbb{R}^P \times \mathbb{R}^P$, los embedding posicionales relativos codifican la distancia (con signo) $r = j - i \in \mathbb{Z}$.
- Para esto, se pueden redefinir las proyecciones iniciales como

$$q_i = W_Q x_i, \quad k_j = W_K(x_j + p_r^k), \quad v_j = W_V(x_j + p_r^v),$$

donde $p_r^k, p_r^v \in \mathbb{R}^D$ son vectores de embedding (entrenables) asociados a la distancia relativa entre los tokens, $r = j - i$.

- Una variante importante es **rotary positional embedding (RoPE)**, la cual combina posición relativa con embeddings sinusoidales.

Mecanismos de normalización

Layer normalization

Layer normalization normaliza (en media y varianza) cada vector de embedding individualmente:

$$\text{LayerNorm}(x) := \alpha \odot \tilde{x} + \beta, \quad \text{donde} \quad \tilde{x} = \frac{x - \text{mean}(x)}{\text{std}(x)}$$

La ubicación de esta capa de normalización dentro de los bloques MHSA y FF influye en la estabilidad del entrenamiento:

- **Post-LN:** es de la forma $x \mapsto \text{LayerNorm}(x + \text{SubLayer}(x))$
- **Pre-LN:** es de la forma $x \mapsto x + \text{SubLayer}(\text{LayerNorm}(x))$

Hoy en día se prefiere la arquitectura Pre-LN ya que genera una dinámica de entrenamiento más estable.

Mecanismos de normalización

Otra técnica de normalización usual es **RMS normalization**, la cual calcula, para cada vector de embedding $x \in \mathbb{R}^D$:

$$\text{RMSNorm}(x) = \alpha \odot \frac{x}{\text{rms}(x)}, \quad \text{donde} \quad \text{rms}(x) = \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2}$$

- Notar que $\alpha \in \mathbb{R}^D$ es el único parámetro entrenable.
- RMSNorm normaliza todos los embeddings a una misma esfera de radio $\frac{1}{\sqrt{D}}$, la cual luego puede deformarse gracias al parámetro de escala $\alpha \in \mathbb{R}^D$.
- Este método de normalización se ha vuelto usual debido a que se puede computar eficientemente.

Mecanismos de atención

Atención aditiva y multiplicativa

Dada la importancia del mecanismo de atención, encontrar otras maneras definirlo puede mejorar considerablemente la eficiencia y rendimiento de un modelo.

Para un vector query $q_i \in \mathbb{R}^P$ y un vector key $k_j \in \mathbb{R}^P$, se podría definir el score de atención, $s_{ij} \in \mathbb{R}$, de dos formas naturales:

- **Atención aditiva:** $s_{ij} = W_0 \tanh(W_1 q_i + W_2 k_j)$.
- **Atención multiplicativa:** $s_{ij} = q_i^\top W k_j$.

Scaled dot-product attention (atención estándar) equivale a fijar $W = \frac{1}{\sqrt{P}} I_P$ en la atención multiplicativa.

La división por $\sqrt{P}$ se realiza para que $\text{Var}\left(\frac{\langle q_i, k_j \rangle}{\sqrt{P}}\right) = 1$, lo que permite controlar los logits que entrarán a la función softmax.

Proyecciones en el mecanismo de atención

Por otro lado, las 3 proyecciones iniciales $x_t \mapsto Wx_t$ realizadas en el cálculo de los vectores $q_t, k_t, v_t \in \mathbb{R}^P$ se utilizan para:

- **Evitar $A \in \mathcal{M}_{T,T}(\mathbb{R})$ diagonal:** si se definiera $s_{ij} = \frac{\langle x_i, x_j \rangle}{\sqrt{P}}$, entonces cada token siempre se colocaría más atención a sí mismo.
- **Proyecciones especializadas:** usar tres proyecciones distintas permite que los vectores $q_t, k_t, v_t \in \mathbb{R}^P$ se puedan adaptar a su rol dentro del mecanismo de atención.
- **Reducir dimensión:** dado que $P \ll D$, las proyecciones permiten realizar el cálculo de atención sobre matrices más pequeñas.
- **Añadir parámetros:** las matrices de proyección son los únicos parámetros entrenables dentro de un cabezal de atención.

Atención cruzada

El mecanismo de auto-atención puede ser extendido a un mecanismo de **atención cruzada**, donde una secuencia de embeddings, $(x_1, \dots, x_T) \subset \mathbb{R}^D$ coloca atención sobre otra secuencia de embeddings, $(\tilde{x}_1, \dots, \tilde{x}_{\tilde{T}}) \subset \mathbb{R}^{\tilde{D}}$. Para esto:

- Se calculan los vectores query como $q_i = W_Q x_i \in \mathbb{R}^P$ ($i \in \{1, \dots, T\}$) usando la primera secuencia.
- Se calculan los vectores key y value como $k_j = W_K \tilde{x}_j \in \mathbb{R}^P$ y $v_j = W_V \tilde{x}_j \in \mathbb{R}^P$ ($j \in \{1, \dots, \tilde{T}\}$) usando la segunda secuencia.
- Se aplica el mecanismo de atención usual.

Notar que no es necesario que $T = \tilde{T}$ ni que $D = \tilde{D}$.

Mixture of experts

Mixture of experts

Luego de un bloque MHSA es usual aplicar un bloque FF ($x_t \mapsto \text{FF}(x_t) = W_2 \phi(W_1 u_t)$), el cual procesa individualmente cada embedding $x_t \in \mathbb{R}^D$ obtenido en el bloque MHSA.

Una extensión es el mecanismo de **mixture of experts (MoE)**:

- Se considera un conjunto de $M \geq 1$ expertos, $\{\text{FF}_1, \dots, \text{FF}_M\}$, cada uno con sus propios parámetros.
- Así, la salida es una combinación convexa de los expertos, donde los ponderadores son determinados por un **enrutador**:

$$\text{MoE}(x_t) = \sum_{m=1}^M \pi_m(x_t) \text{FF}_m(x_t) \quad \text{donde} \quad \pi(x_t) = \text{softmax}(W_r x_t)$$

- Para reducir el costo computacional se usa **top-K gating**, donde solo los $K \ll M$ expertos más probables procesan cada x_t .

Otros componentes de la arquitectura Transformer

Los otros dos componentes de la arquitectura estudiada son las **conexiones residuales** y el uso de **dropout**:

- Las conexiones residuales $x \mapsto x + \text{Bloque}(x)$ promueven gradientes más estables a lo largo de todas las capas, por lo que ayudan a mitigar el vanishing gradient problem.
- Más aún, se ha demostrado empíricamente que las conexiones residuales promueven una función objetivo más *plana*.
- Es común aplicar dropout a capas que tienen muchos parámetros ya que son más propensas al overfitting.
- Hoy en día muchos modelos no utilizan dropout.

En la próxima clase.

- Otros modelos tipo Transformer (BERT, ViT, Chameleon, CLIP).
- LLMs (chat templates, leyes de escala, habilidades emergentes, algunos LLMs famosos).

Modelos Generativos Profundos

Clase 7: Modelos autorregresivos y LLMs (parte III)